

M1 informatique, M2 SAAD
TP. ETL
Unité YSMIN1A1 : Bases de données avancées

L'objectif de ce TP est de concevoir un *datawarehouse* et les processus d'ETL pour une application particulière, dont les données sont fournies. Après avoir récupéré ces données, il s'agira donc de concevoir le *datawarehouse*, d'en implémenter la structure dans un serveur de bases de données relationnelles, puis d'écrire des *pipelines* Apache Hop permettant de l'alimenter à partir des données fournies.

Au final, il faudra donc : rédiger un petit document décrivant les choix faits pour le *datawarehouse*; créer des scripts SQL permettant d'implémenter sa structure; créer un projet Apache Hop (qui sera constitué de plusieurs *pipelines*).

1 Données fournies

On s'intéresse aux données d'une entreprise (fictive) qui vend des prestations (dépannage informatique, mise en service internet, dépannage TV, etc.) à des particuliers. Ces prestations correspondent à des interventions de techniciens, et elles ont donc lieu à une certaine date, à une certaine heure, et à une certaine adresse. Plus précisément, les données concernant ces ventes sont disponibles dans une table nommée « Ventes » et ayant pour schéma :

```
`heure_debut` TEXT NOT NULL,  
`heure_fin` TEXT NOT NULL,  
`num_client` INTEGER NOT NULL REFERENCES Clients(num_client),  
`code_prestation` TEXT NOT NULL,  
`prix` REAL NOT NULL,  
`adresse` TEXT NOT NULL,  
`code_postal` TEXT NOT NULL,  
`ville` TEXT NOT NULL,  
'date_insertion' TEXT NOT NULL
```

Les numéros des clients (`num_client`) sont des clefs étrangères, vers la table nommée « Clients », qui a pour schéma :

```
`num_client` INTEGER PRIMARY KEY,  
`nom` TEXT NOT NULL,  
`prenom` TEXT NOT NULL,  
`adresse` TEXT NOT NULL,  
`code_postal` TEXT NOT NULL,  
`ville` TEXT NOT NULL
```

Ces deux tables sont fournies dans une base de données SQLite, nommée `operational_data.db`.

La colonne `code_prestation` de la table **Ventes** fait référence au code de la prestation, tel que défini dans le fichier CSV `prestations.csv` fourni, qui donne la liste des prestations avec leur code, leur nom, et leur catégorie (« Dépannage » ou « Installation »). À noter, les prestations n'ont pas de prix en tant que tel : deux ventes d'une même prestation peuvent très bien avoir des prix différents (cela peut dépendre de la difficulté du dépannage, ou encore des consommables utilisés, par exemple).

Enfin, dans les deux tables les adresses sont représentées par trois colonnes; par exemple, l'adresse « 1 rue de la République, 14000 Caen » est découpée en « 1 rue de la République » (`adresse`), « 14000 » (`code_postal`), et « Caen » (`ville`). Dans la table **Ventes**, il s'agit de l'adresse à laquelle la prestation a

été réalisée, tandis que dans la table **Clients**, il s'agit de l'adresse du client (qui a pu payer une prestation ayant été réalisée ailleurs qu'à son domicile).

On considérera qu'un client peut changer d'adresse au cours du temps. En revanche, on considérera la liste des villes, codes postaux, etc., ainsi que le catalogue des prestations, comme stables.

2 Attendus

L'objectif final du traitement de ces données sera de pouvoir les restituer selon, au moins, les axes suivants :

- prix de vente, chiffre d'affaires, nombre de ventes, etc.,
- durée moyenne d'intervention, durée maximum d'intervention, etc.,
- analyse par région, code postal, département, etc., du lieu de réalisation des prestations (avec la ville comme plus petite granularité),
- analyse selon les clients, leur ville, leur code postal, leur département, etc.,
- analyse par heure de la journée, par jour de la semaine, du mois, par trimestre, etc.,
- analyse par type et catégorie de prestations.

3 Création d'un entrepôt de données

Pour les deux questions qui suivent, résumer la modélisation choisie, et justifier brièvement les choix, dans un petit document (2 ou 3 pages).

Question 1. *Proposer une modélisation en étoile des données, en prenant en compte les axes d'analyse décrits ci-dessus.*

Indication : *On pourra répéter les données correspondant aux villes, départements, régions, etc. pour les lieux de réalisation des prestations et pour les lieux de résidence des clients.*

Question 2. *Déduire de la modélisation un schéma relationnel pour un entrepôt de données. Veiller à insérer les éléments nécessaires pour que les données soient résilientes aux changements (adresses des clients), et pour permettre la traçabilité des données.*

Indication : *On pourra utiliser directement les chaînes datetime comme clefs pour les dates/heures ; en effet, une telle chaîne identifie par définition une unique date/heure, et elles sont au même format (yyyy-MM-dd HH:mm) dans la table des ventes (champs `heure_debut` et `heure_fin`) et dans le fichier de dates fournis (voir exercice 4).*

Exercice 3. *Écrire les scripts SQL permettant de créer les tables correspondant à votre modélisation dans une base de données SQLite. Exécuter ces scripts et les conserver.*

Le datawarehouse est désormais prêt à recevoir les données.

4 Alimentation de l'entrepôt

L'objectif de cette partie est d'alimenter l'entrepôt créé avec les données. Le répertoire dans lequel sont stockées les données fournies sur la page du cours jouera ici le rôle de *staging area*.

Pour les exercices qui suivent, créer un projet Apache Hop, qui accueillera tous les *pipelines* demandés. Utiliser au maximum les métadonnées et les environnements, afin que les changements de noms de fichiers, de chemins, etc., puissent être répercutés simplement dans le projet.

Note La partie A fournit quelques indications sur les transformations Apache Hop à utiliser.

Exercice 4. *Créer un pipeline pour alimenter la dimension correspondant aux dates et celle correspondant aux lieux de réalisation des prestations.*

Indication : *On considère ces dimensions comme conformes. Pour simuler cela, leur contenu est fourni dans les fichiers `dates.csv` et `geography.csv`. Il n'y a donc plus qu'à écrire des pipelines qui lisent ces données (éventuellement en filtrant les colonnes), et les insèrent dans le datawarehouse. Les dates sont volontairement limitées (de 2019 à 2021) pour ne pas surcharger Apache Hop pendant le TP.*

Indication : *Parmi les données fournies dans le fichier `geography.csv`, on trouve des codes INSEE et des codes postaux vides. Les lignes correspondantes peuvent être sorties dans un fichier d'erreurs : aucune vente n'a lieu, et aucun client n'habite, dans les communes en question.*

Note La gestion des dates est volontairement simplifiée. Pour une gestion réelle, dans la dimension conforme les dates seraient uniquement identifiées par une date UTC. On devrait alors lire les dates des données opérationnelles en connaissant leur fuseau horaire et les convertir en UTC. Avec la gestion simplifiée, certaines erreurs peuvent se produire, par exemple pour les dates correspondant aux changements d’heures d’été/d’hiver.¹

Exercice 5. Créer un pipeline pour alimenter les autres dimensions du datawarehouse, en les transformant autant que rendu nécessaire par la structure de votre datawarehouse.

Indication : Pour la dimension des clients, on pourra simplifier le pipeline en considérant que, puisqu’il s’agit d’une alimentation initiale :

- les adresses des clients sont valides depuis une date inconnue et jusqu’à nouvel ordre (donc en laissant à `null` les deux dates de validité) ;
- le numéro du client peut être utilisé à la fois comme business key et comme surrogate key, puisque c’est une clef primaire de la table `Clients`.

Exercice 6. Créer un pipeline permettant d’alimenter la table de faits, à partir des données de la table `Ventes`, en les transformant à nouveau autant que nécessaire.

A Aide technique pour Apache Hop

Les principales transformations à utiliser sont les suivantes. D’autres peuvent bien entendu être utilisées, notamment pour enrichir les traitements.

Extraction de données

- **CSV file input**, pour les fichiers délimités, en prenant garde à modifier les types devinés par la transformation (par exemple, à spécifier les codes postaux comme étant de type `String`).
- **Table input**, pour les bases de données, en utilisant par exemple le bouton *Get SQL select statement* (après sélection de la *Connection*).

Transformation de données

- **Sort rows**, pour trier les données, en spécifiant les champs sur lesquels baser le tri (par priorité décroissante).
- **Merge join**, pour effectuer des jointures, en prenant garde de trier les données en amont, par ordre croissant sur les clefs de jointure ; pour ce composant, les champs à spécifier sont ceux sur lesquels doit s’effectuer la jointure, et le type *LEFT OUTER* signifie que toutes les lignes de la *First transform* sont conservées, qu’il y ait ou non une ligne correspondante dans la *Second transform*.
- **Data validator**, pour vérifier le format des données ; deux *hops* au moins doivent « sortir » de ce composant, un pour les lignes valides, et un pour les lignes non valides (*Error handling of transform*).
- **Select values**, pour la conversion des chaînes *datetime* du type *String* au type *Date*, et vice-versa ; pour cela, utiliser l’onglet *Meta-data* avec les champs à convertir, leur nouveau type, et le format ; attention, il ne faut pas renseigner en même temps la colonne *Rename to*, car la conversion serait alors ignorée.
- **Calculator**, pour calculer la durée des prestations et la stocker dans un nouveau champ via le *Calculation* « Date A - Date B (minutes) ».

Chargement de données dans le *datawarehouse*

- **Table output**, en spécifiant la *Connection* et la *Target table*, et en prenant soin de cocher *Truncate table* pour que la table soit vidée avant l’exécution de la transformation (ce qui fait sens puisque l’on opère une alimentation initiale) ; pour spécifier les champs à enregistrer, cocher *Specify database fields* puis utiliser l’onglet *Database fields* et le bouton *Enter field mapping* pour spécifier quel champ des données (*Stream field*) doit être enregistré dans quelle colonne de la table (*Table field*) ; les colonnes de la table spécifiées comme *AUTOINCREMENT* ou ayant une valeur par défaut peuvent ne pas être associées à des champs des données.

1. Si de telles erreurs se produisent dans le *pipeline*, les ignorer.